For a byte-level analysis of string data, the `count_chars()` function will meet a number of use cases. It operates in five different modes and will return either an array or string depending on the mode parameter passed in when called. Its job is to simply report on byte values found or not found in the entire string being analyzed; it is not aware of character encodings and therefore will not provide information about multi-byte characters (it should probably have been called `count_bytes()`, but this function comes from a time and place where *byte* and *character* were synonymous).

```
var_dump(count_chars("abcd"));
```

```
Output:
Array
(
    [0] => 0
    ...
    [96] => 0
    [97] => 1
    [98] => 1
    [99] => 1
    [100] => 1
    [101] => 0
    ...
```

By default, `count_chars()` outputs a 256-element array of integers; each element position corresponds to a byte value count. The output above shows that bytes *97–100* (remember these are decimal values) each have a count of one. Every other byte has a count of zero. Our input string contained the ASCII letters `"abcd"`, which correspond to those points in the ASCII codepage (the letter `"a"` is ASCII byte *0x61*, which has a decimal value of *97*, and so on). Remember that we can also pass an extra (integer) argument to the function to control its behavior:

```
print_r(count_chars("abcd", 1));
```

```
Output:
Array
(
    [97] => 1
    [98] => 1
```